

Foundations for
programming languages
9: Lambda calculus

Isao Sasano

Lambda calculus

Lambda calculus was invented in 1930's (before physical digital computers were made). Nowadays it becomes a core of functional languages. It covers all the functions that are computable.

[Lambda expressions]

$\langle t \rangle ::= \langle c \rangle$	constants
$\langle \text{id} \rangle$	variables
$(\lambda \langle \text{id} \rangle . \langle t \rangle)$	lambda abstractions
$(\langle t \rangle \langle t \rangle)$	function applications

$\langle c \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid \dots$

$\langle \text{id} \rangle ::= x \mid y \mid z \mid w \mid \dots$

Lambda expressions

Conventions when writing lambda expressions:

1. Function applications associate to the left, so that $t_1 t_2 t_3$ should be read as $(t_1 t_2) t_3$.
2. The scope of each variable bound in lambda abstraction is interpreted as being as large as possible, so an expression $\lambda x. \dots$ may be parenthesized by inserting a left parenthesis after the dot and the matching right parenthesis as far to the right as will produce a syntactically well-formed expression.
3. When parentheses are omitted, obey 1 and 2.

(ex.) We read $x y z$ as $((x y) z)$.

(ex.) We read $\lambda x. \lambda y. \lambda z. x y z$ as $(\lambda x. (\lambda y. (\lambda z. ((x y) z))))$.

Lambda expressions

Lambda expressions are invented while pursuing the essentials of computations.

Intuitively a lambda abstraction is a form for writing a function. For example, the function g defined by

$$g(x) = x + 1$$

is written using lambda abstraction as follows.

$$g = \lambda x. x + 1$$

Lambda calculus

Lambda calculus is a computation system according to the idea where computation is a series of transformations (called beta reductions) of lambda expressions.

Examples of beta reductions

(ex.) $(\lambda x. x) z \rightarrow z$

(ex.) $(\lambda y. y) z \rightarrow z$

Intuitively the expression $(\lambda x. x)$ is a function that takes an argument and returns it, so even if we change the x 's to y 's, the meaning does not change.

Similarly, $g(x) = x$ and $g(y) = y$ have the same meaning in mathematics.

Examples of beta reductions (cont.)

$$(ex.) (\lambda x. x y) (\lambda z. z) \rightarrow (\lambda z. z) y$$

$(\lambda x. x y)$ takes an argument x and applies x to y . In this example, $(\lambda z. z)$ is the actual argument, so it is applied to y . (This can further be beta reduced to y .)

$$(ex.) (\lambda x. x) (\lambda y. y) \rightarrow (\lambda y. y)$$

$(\lambda x. x)$ represents a function that takes an argument x and returns x . In this example, $(\lambda y. y)$ is the actual argument, so the result is $(\lambda y. y)$.

Meta variables

We use the following meta variables.

M, N, P, M₁, M₂, etc: lambda expressions

c: constants (natural numbers)

x, y, z : variables (*x, y, z, w*, etc. In this slide, meta variables are written in italic and usual variables are written in roman.)

Beta reductions

Beta reduction is written by $\rightarrow$ and defined as follows.

- $(\lambda x. M) N \rightarrow M [N / x]$
- If $M \rightarrow N$,
 $\lambda x. M \rightarrow \lambda x. N$
 $MP \rightarrow NP$
 $PM \rightarrow PN$

$M [N / x]$ (substitution) represents a lambda expression obtained by replacing all free occurrences of x in M with N . Note that bound variables are renamed when necessary.

An example

The previous example $(\lambda x. x) z$ is beta reduced to $x [z / x]$. $x [z / x]$ represents a lambda expression obtained by replacing x with z , namely z .

The lambda expression $(\lambda x. x y) (\lambda z. z)$ is beta reduced to $(x y) [(\lambda z. z) / x]$. $(x y) [(\lambda z. z) / x]$ represents a lambda expression obtained by replacing all occurrences of x in $(x y)$ with $(\lambda z. z)$, namely $(\lambda z. z) y$.

Another example

Notice $(\lambda x. y) [x / y]$ does not represent $(\lambda x. x)$.
 x is local variable in $(\lambda x. y)$, so it is not the same as x in $[x / y]$. So we rename the bound variable x in $\lambda x. y$ with a new name such as z and obtain $\lambda z. y$.
Then we replace y in $\lambda z. y$ with x and obtain $\lambda z. x$.
We define substitution in the next page with taking into account the name collision.

The definition of $M [N / x]$

When M is a constant:

$$c [N / x] = c$$

When M is a variable:

$$x [N / x] = N$$

$$y [N / x] = y \quad (x \neq y)$$

When M is a lambda abstraction:

$$(\lambda y. M) [N / x] =$$

$$\left\{ \begin{array}{ll} \lambda y. M & (x = y) \\ \lambda y. (M [N / x]) & (x \neq y, y \notin \text{FV}(N)) \\ \lambda z. ((M [z / y]) [N / x]) & (x \neq y, y \in \text{FV}(N), z \neq x, \\ & z \notin \text{FV}(M), z \notin \text{FV}(N)) \end{array} \right.$$

When M is a function application:

$$(M_1 M_2) [N / x] = (M_1 [N / x]) (M_2 [N / x])$$

Free variables

Free variables in lambda expression M , written as $FV(M)$, is defined as follows.

$$FV(c) = \{ \}$$

$$FV(x) = \{ x \}$$

$$FV(\lambda x. M) = FV(M) \setminus \{ x \} \text{ (}\setminus\text{ is the operation for taking the difference of two sets.)}$$

$$FV(M_1 M_2) = FV(M_1) \cup FV(M_2)$$

Examples

(ex.) Free variables in a lambda expression $(\lambda x. x)$:

$$FV (\lambda x. x) = FV (x) \setminus \{x\} = \{x\} \setminus \{x\} = \{ \}$$

So $(\lambda x. x)$ has no free variable.

(ex.) Free variables in a lambda expression $(\lambda x. x y)$:

$$\begin{aligned} FV (\lambda x. x y) &= FV (x y) \setminus \{x\} = (FV (x) \cup FV(y)) \setminus \{x\} \\ &= (\{x\} \cup \{y\}) \setminus \{x\} = \{x, y\} \setminus \{x\} = \{y\} \end{aligned}$$

So $(\lambda x. x y)$ has just one free variable, y .

Exercise 1

- (1) Obtain the free variables in $(\lambda z. z) w$ according to the definition of free variables.
- (2) Obtain the free variables in $(\lambda z. z y) ((\lambda z. z) w)$ according to the definition of free variables.

Exercise 2

- (1) What does $(x\ y)\ [z/x]$ represent?
- (2) What does $(\lambda y. x\ y)\ [z/x]$ represent?
- (3) What does $(\lambda y. x\ y)\ [y/x]$ represent?
- (4) What does $(\lambda y. x\ y)\ [\lambda z. z\ y/x]$ represent?

Exercise 3

- (1) Beta reduce once $(\lambda x. x y) (\lambda z. z)$.
- (2) Beta reduce once $(\lambda x. (\lambda y. x y)) (\lambda z. y z)$.

Sequences of beta reductions

When M_1 is beta reduced more than or equal to 0 times to M_n ,

$$M_1 \rightarrow M_2 \rightarrow \dots \rightarrow M_n$$

is called a sequence of beta reductions.

(This can also be written as $M_1 \rightarrow^* M_n$.)

When $M \rightarrow N$ and $N \rightarrow M$, we write $M \leftrightarrow N$.

An example

A lambda expression $(\lambda x. \lambda y. x y) z w$ can be beta reduced to $z w$ as follows.

$$\begin{aligned} & (\lambda x. \lambda y. x y) z w \\ & \rightarrow (\lambda y. z y) w \\ & \rightarrow z w \end{aligned}$$

Exercise 4

A lambda expression $(\lambda x. \lambda y. x y) (\lambda z. z) w$ can be transformed to w by applying beta reductions. Write each step of beta reductions.

Church-Rosser theorem

If $M \rightarrow^* M_1$ and $M \rightarrow^* M_2$ then there exists a lambda expression N such that $M_1 \rightarrow^* N$ and $M_2 \rightarrow^* N$.

An example

A lambda expression $(\lambda x. \lambda y. x y) (\lambda z. z) w$ is beta reduced as follows.

$$(\lambda x. \lambda y. x y) (\lambda z. z) w \rightarrow (\lambda y. (\lambda z. z) y) w$$

There are two ways to beta reduce the obtained lambda expression.

$$(1) (\lambda y. (\lambda z. z) y) w \rightarrow (\lambda y. y) w$$

$$(2) (\lambda y. (\lambda z. z) y) w \rightarrow (\lambda z. z) w$$

By applying beta reduction once more, we obtain w .

As this example shows, when a lambda expression is beta reduced to two different lambda expressions, they can be beta reduced to the same lambda expression.

Exercise 5

Beta reduce the lambda expression $(\lambda x. \lambda y. x y) (\lambda x. x y) w$ until obtaining a lambda expression that can not be beta reduced.

In this example, there are two sequences of beta reductions. Show both of them.

References

Alonzo Church,
“An unsolvable problem of elementary number theory”,
American Journal of Mathematics,
vol. 58 , pp. 345-363, 1936.